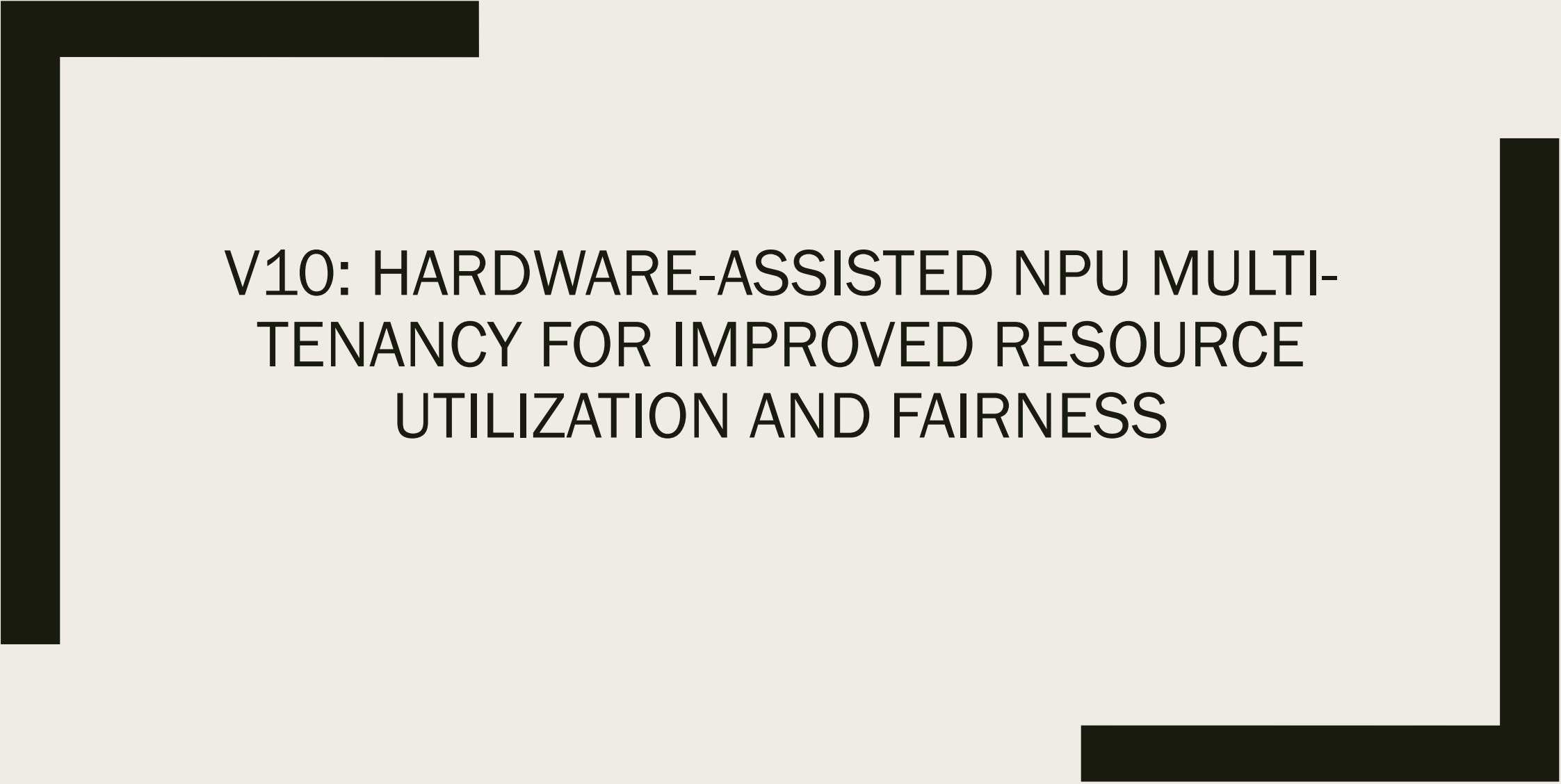




V10: HARDWARE-ASSISTED NPU MULTI-TENANCY FOR IMPROVED RESOURCE UTILIZATION AND FAIRNESS

목차

- 1. Abstract
- 2. Introduction
- 3. Characterization Study of NPU Utilization(다음 시간...)
- 4. Design and Implementation
- 5. Evaluation

Abstract

Abstract

- 현대 클라우드 플랫폼은 ML 추론 서비스 가속화를 위해 Google Cloud TPU와 같은 NPU를 배치함 + 여러 ML 애플리케이션이 동일한 NPU를 공유할 수 있도록 시분할(time-multiplexed) 및 선점(preemptive) 기반의 공유 메커니즘 개발
 - 시분할: 시간을 나누어 순서대로 자원 사용
 - 선점: 우선순위가 높은 작업이 들어오면 자원을 가로챈
- 그러나 이러한 기존 방식이 NPU 내에서 미세한 하드웨어 자원 공유를 지원 못함
 - > 활용도가 매우 낮음

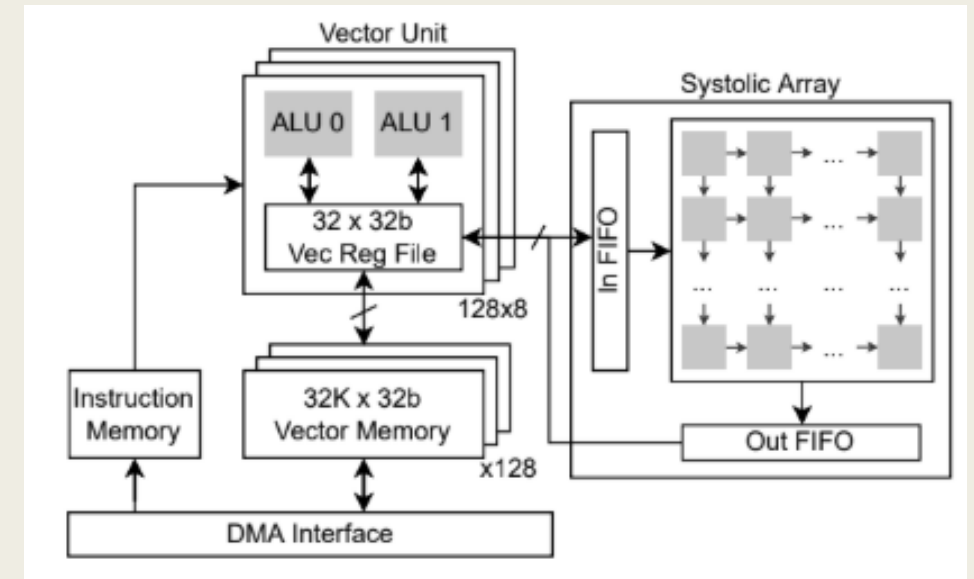
Abstract

- 서로 다른 ML 서비스 간의 공정성을 보장하는 동시에 자원 활용도를 개선하는 하드웨어 보조형 NPU 멀티 테넌시 프레임워크인 V10 제시
- V10
 1. SA와 VU에서 연산자를 동시에 실행할 수 있게 하는 연산자 스케줄러를 채택
 2. 다양한 우선순위 기반 자원 공유 메커니즘을 시행할 수 있는 유연성을 제공
 3. 공유 NPU에서 가장 잘 맞는 ML 서비스 조합을 식별하는 클러스터링 기반 워크로드 배치 메커니즘을 개발
- V10은 최신 NPU multi-tenancy 방식과 비교했을 때 전체 NPU 활용도를 1.64배 개선하고, 총 처리량을 1.57배 증가시키며, ML 서비스의 평균 지연 시간을 1.56배, 꼬리 지연 시간을 1.74배 단축하는 성과를 거둠

Introduction

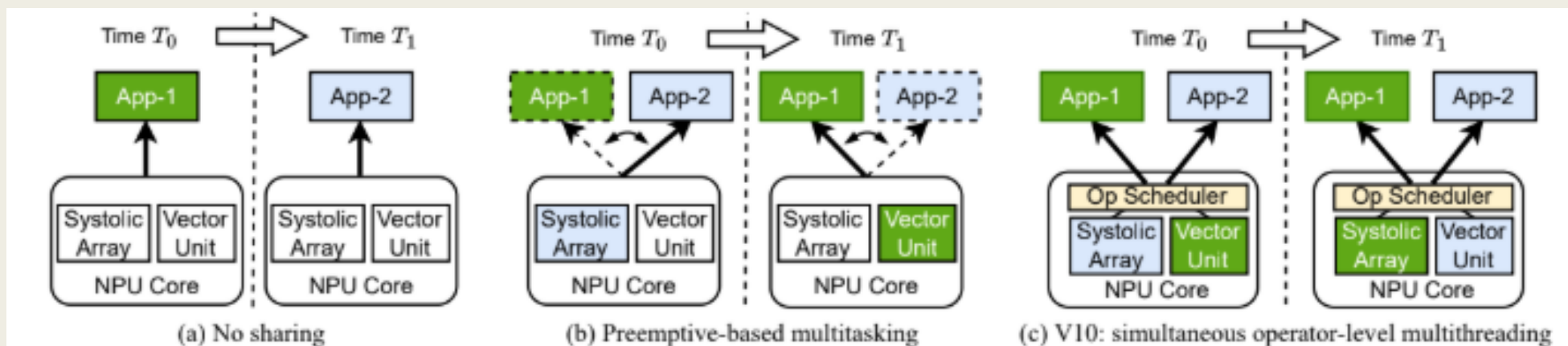
Introduction

- NPU 코어는 일반적으로 행렬 곱셈의 데이터 재사용 패턴을 활용하는 대규모 SA와 기타 일반적인 벡터 연산을 위한 VU으로 구성됨



Introduction

- 클라우드에서 NPU를 사용하는 간단한 방법은 각 NPU 코어를 하나의 ML 서비스에 독점적으로 할당하는 것인데, 이는 자원 공유를 완전히 허용하지 않음(a)
-> 자원 활용도가 가장 낮음
- 또한 현대적인 클라우드 플랫폼이나 PREMA 같은 최신 기술에서 사용하는 방식으로, Time-sharing을 통해 NPU를 공유+ 실행 중인 ML 커널을 중간에 중단(선점)하고 다른 커널을 스케줄링 (b)
-> 한 번에 한 가지 앱만 NPU를 점유 -> 여전히 NPU 내부의 유향 하드웨어 활용x
- 서로 다른 앱의 연산자들을 동시에 실행(c)



Introduction

- NPU 활용도를 이해하기 위해, 우리는 먼저 실제 Google Cloud TPU에 대한 철저한 연구를 수행 후 알게 된 사실들
 1. 대다수의 ML 추론 서비스에서, 다양한 입력 배치 크기를 가진 단일 워크로드는 TPU 코어의 총 가용 FLOPS(초당 부동 소수점 연산)의 절반 미만만을 활용
 2. 이는 주로 ML 워크로드에서 MXU와 VPU의 불균형한 사용 때문
 3. XU와 VPU의 연산자 균형이 맞더라도, 텐서 연산자 간의 데이터 의존성 때문에 두 연산 유닛을 동시에 점유하지 못하는 경우가 많음
 4. 활용도가 낮은 연산 유닛은 추가로 HBM 대역폭의 저조한 활용을 초래
 5. 여러 ML 서비스와 함께 TPU의 시분할을 활성화하고 최신 선점 메커니즘을 적용 하더라도, 연산 활용도를 크게 개선할 수 없음

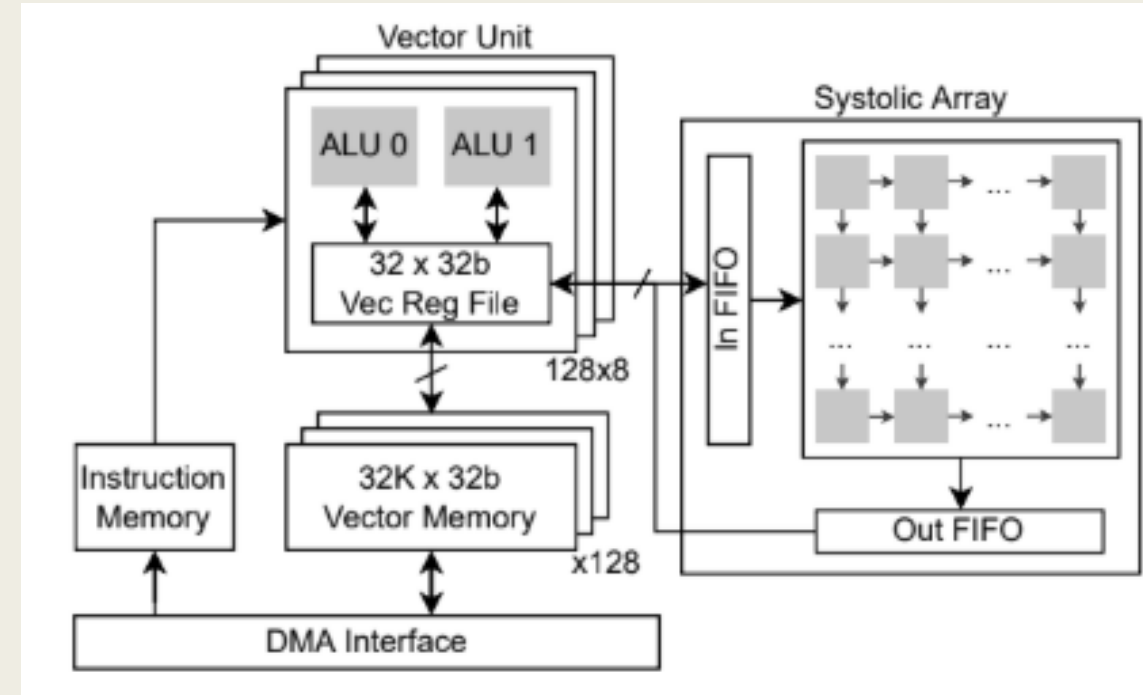
Introduction

- NPU 내부에 연산자 스케줄러를 채용하여 ML 워크로드의 미세한 동시 실행을 가능하게 함. -> v10
- 이 스케줄러는 ML 커널 내에서 SA와 VU의 불균형한 사용으로 인해 발생하는 유휴 사이클을 활용하여 서로 다른 ML 워크로드의 연산자를 동시에 실행할 수 있게 함
- v10 해결책
 1. 동시 실행
 2. 세밀한 선점
 3. 스마트한 짝짓기
- V10이 최신 선점형 멀티태스킹 방식과 비교하여 NPU 활용도를 1.64배 개선하고, 병합된 ML 워크로드의 총 처리량을 1.57배 증가시키며, 각 워크로드의 평균 지연 시간을 1.56배 줄이고, 꼬리 지연 시간을 1.74배 감소시킴

Characterization Study of NPU Utilization

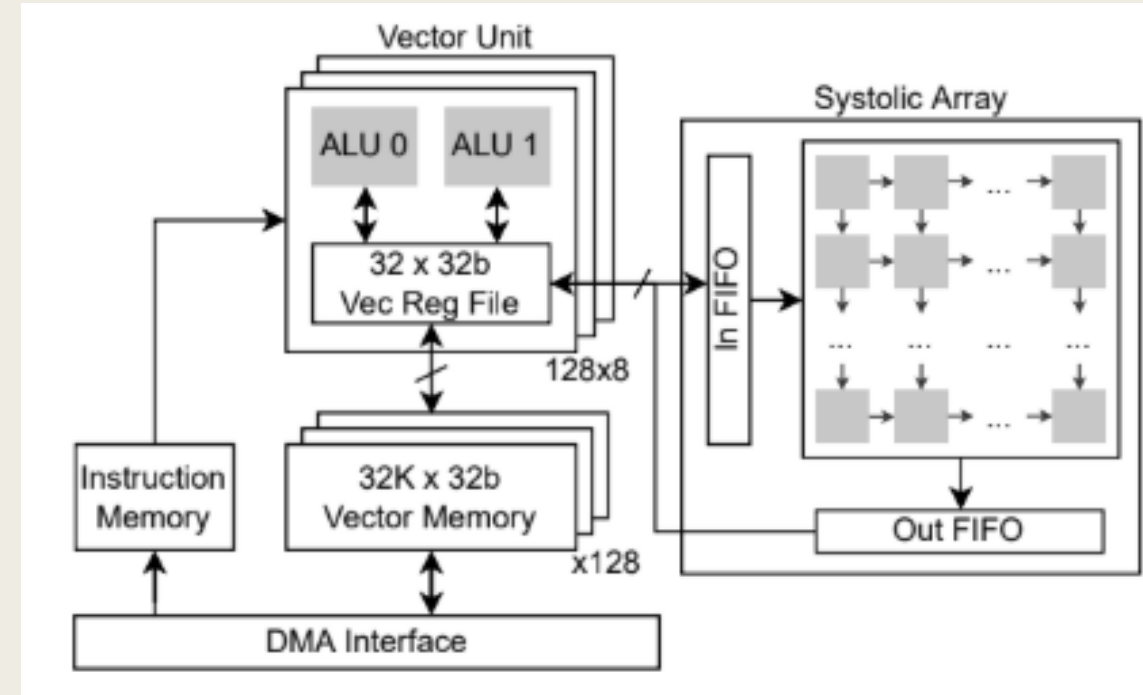
Characterization Study of NPU Utilization

- NPU는 SA, VU, SRAM 버퍼로 구성됨
- DMA (Direct Memory Access): 메인 연산 유닛(SA, VU)의 도움 없이 스스로 HBM에서 SRAM으로 데이터를 옮기는 작업을 수행



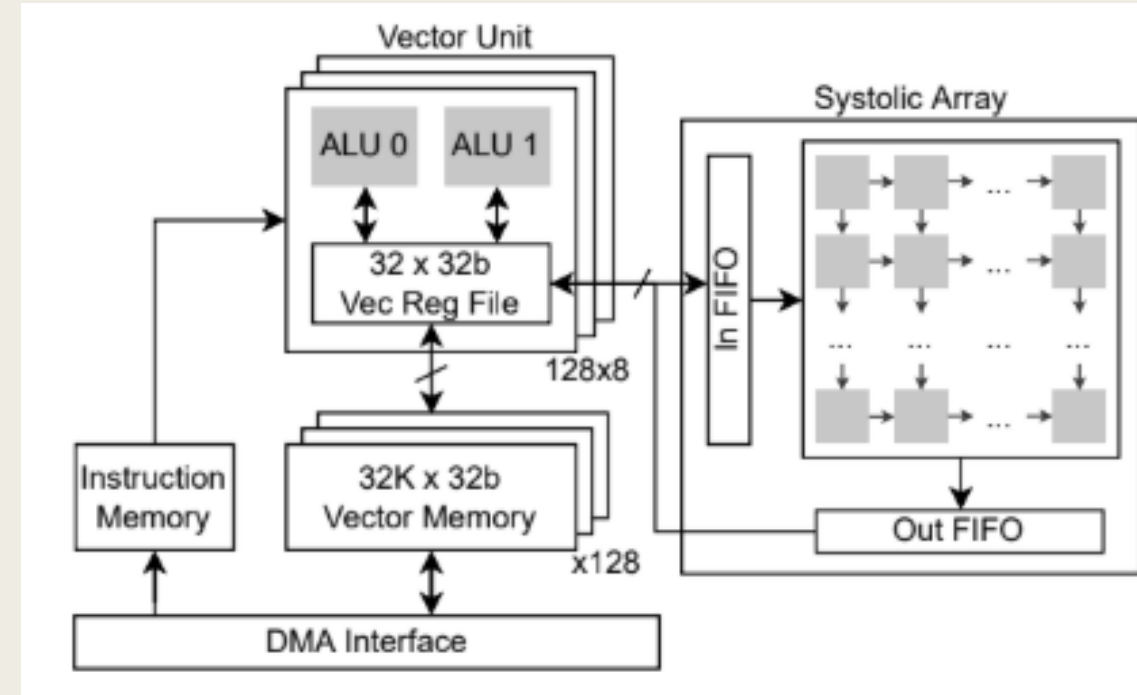
Characterization Study of NPU Utilization

- SIMD 유닛
- 2D 벡터 레지스터 (8×128)
- Load/Store
- FIFO 버퍼



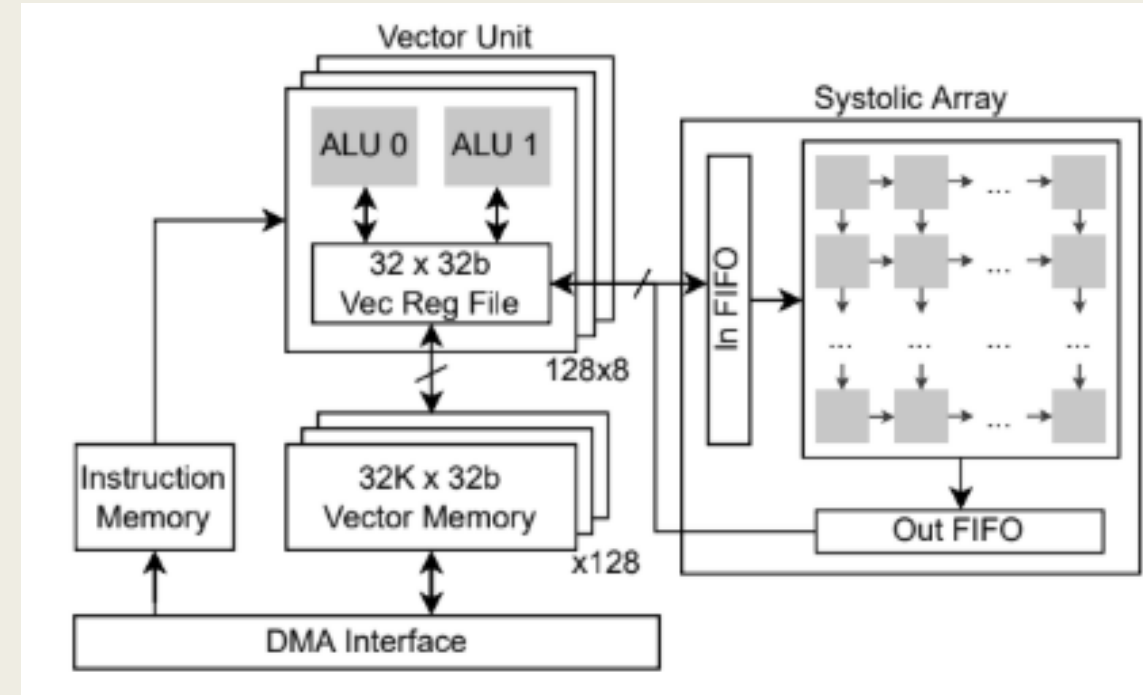
Characterization Study of NPU Utilization

- PE (Processing Element): 아주 작은 계산기들이 128x128 형태로 바둑판처럼 깔려 있음. 각 PE는 매 사이클마다 곱하기와 더하기 수행



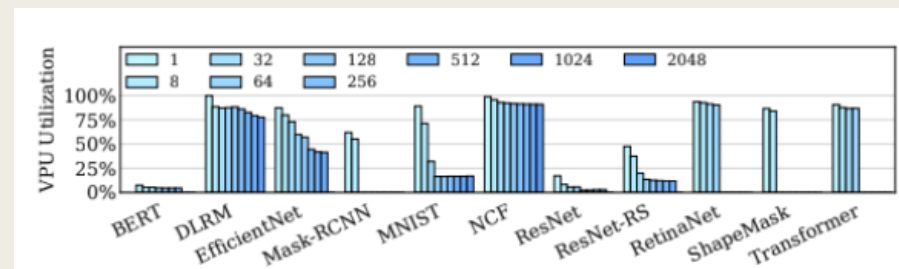
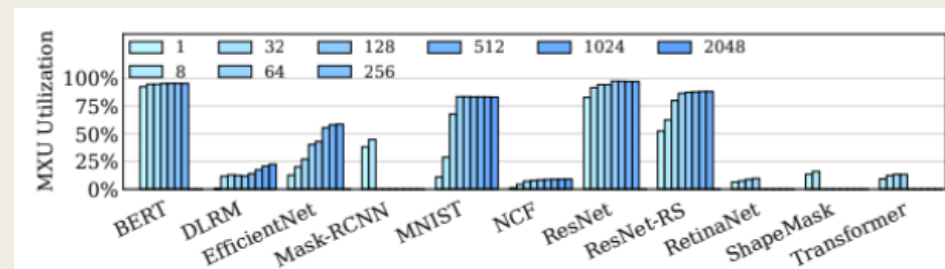
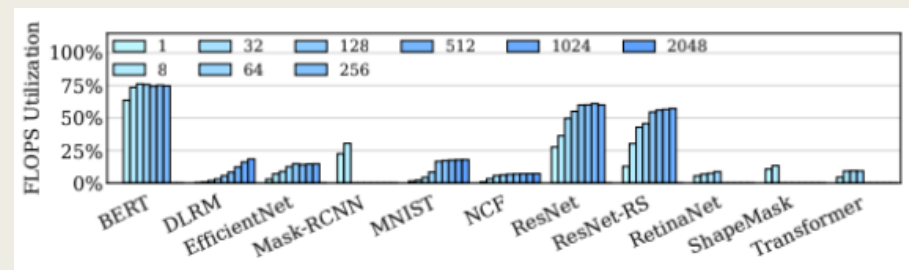
Characterization Study of NPU Utilization

- 컴파일러가 DNN 모델을 분석해 NPU에 내리는 5가지 핵심 명령
- 1. push / pushw %src: 벡터 레지스터에 있는 데이터를 SA로 보냄
- 2. pop %dst: SA의 행렬 계산 후 그 결과물을 가져와 벡터 레지스터에 저장
- 3. ld %dst: 벡터 메모리에 있는 데이터를 벡터 레지스터로 읽어옴
- 4. st %dst: 레지스터에 있는 최종 결과물을 벡터 메모리에 저장
- 5. ALU: 벡터 유닛 안에서 덧셈, ReLU, Sigmoid 같은 요소별 계산 수행



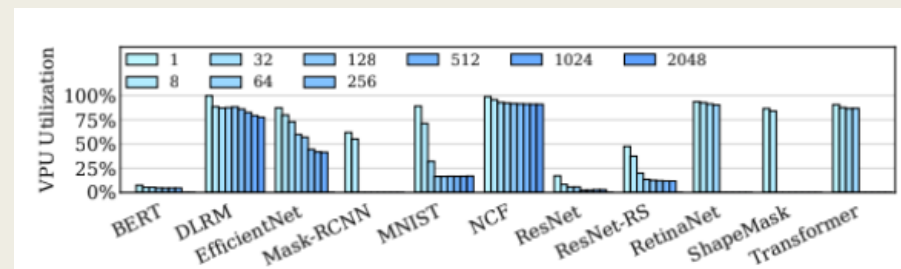
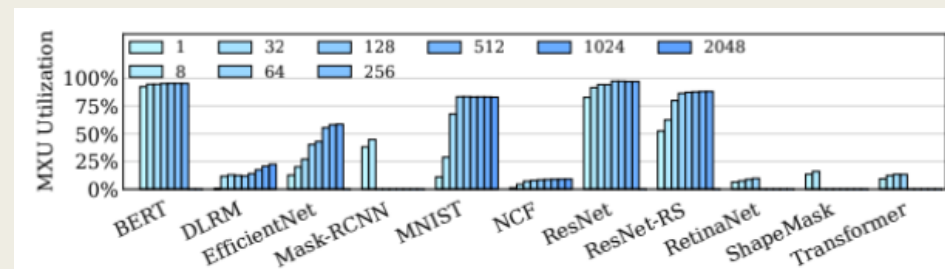
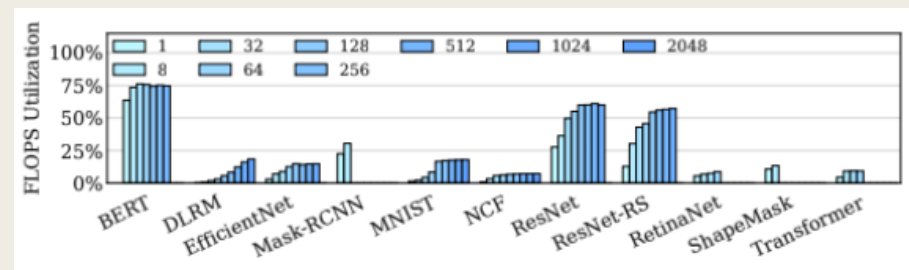
Characterization Study of NPU Utilization

- 대부분의 DNN 워크로드는 TPU 코어의 총 가용 FLOPS의 절반 미만을 활용
-> MXU가 시간적으로 충분히 활용되지 않음
- 이를 통해 관찰된 점
=> MXU와 VPU의 시간적 유휴 상태로 인해 TPU 코어의 계산 자원이 상당히 저조하게 활용되고 있음



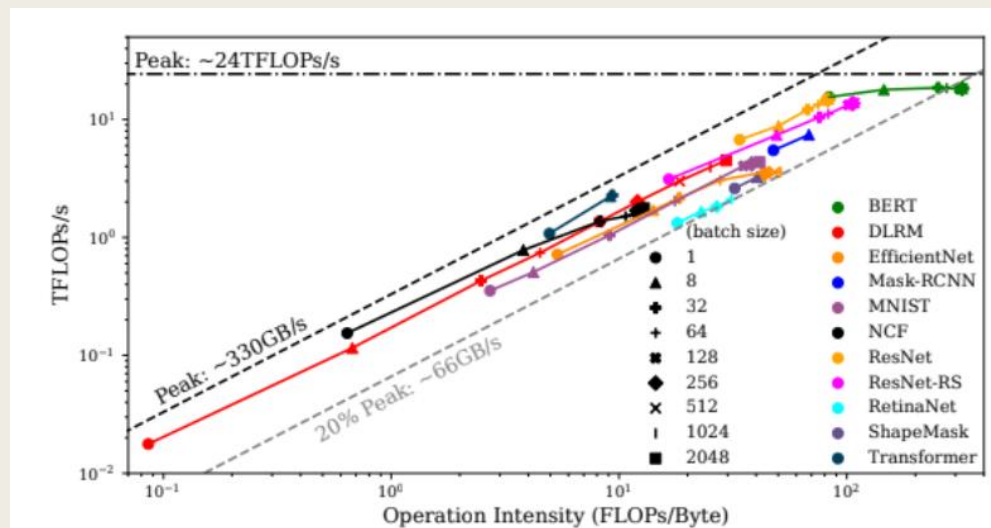
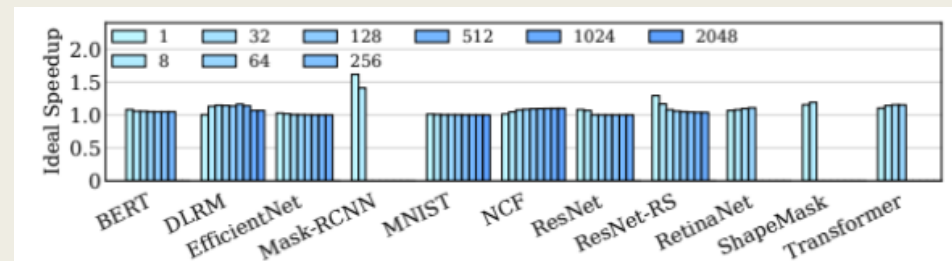
Characterization Study of NPU Utilization

- 이러한 저활용을 해결하기 위해 컴파일러 최적화를 통해 MXU와 VPU의 실행을 중첩시키는 것
-> 이러한 이점도 제한적
 1. 데이터 의존성이 연산자 수준의 병렬성을 제한
 2. 연산자 간 데이터 의존성이 없더라도 병렬성은 MXU와 VPU의 불균형한 사용으로 인해 여전히 제한됨
- 관찰2: MXU와 VPU의 시간적 유휴 상태는 데이터 의존성과 불균형한 사용으로 인한 제한된 연산자 수준 병렬성 때문에 발생. 이러한 제한은 모델 아키텍처의 고유한 특성이므로 단일 워크로드의 활용도를 컴파일러 최적화로 더 높이는 어려움



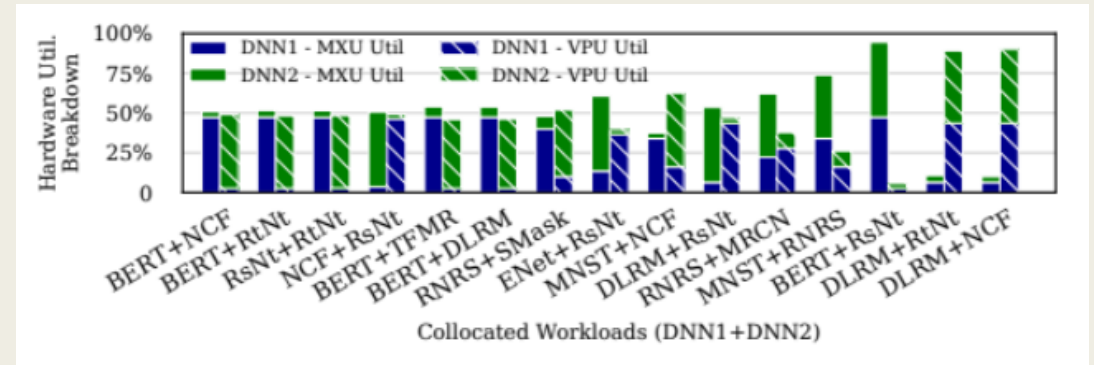
Characterization Study of NPU Utilization

- 배치 크기가 커질수록 계산 집약도가 높아져 데이터 재사용이 많아지므로 대역폭 활용도는 감소
- 계산 및 메모리 집약도 사이의 상관관계 확인
- 관찰 3: 오프칩 HBM 대역폭은 NPU의 FLOPS 저조 활용의 결과로 함께 저조하게 활용됩니다. 이는 HBM이 SaA의 피크 성능에 맞춰 설계되었기 때문



Characterization Study of NPU Utilization

- 선점형 멀티태스킹은 작업 수준에서 워크로드를 선점하여 시간을 나누어 씀
- 관찰 4: 최신 선점 기반 방식도 여전히 낮은 NPU 활용도와 불균형 문제로 고통받고 있음, 무분별한 워크로드 병합은 자원 경합을 일으켜 활용도 개선을 어렵게 함



Evaluation

Evaluation

- 전체 NPU 활용도를 1.64배 향상시킴
- 통합 처리량(throughput)을 1.57배 증가시킴
- ML 서비스의 평균 지연 시간(latency)을 1.56배, 꼬리 지연 시간(tail latency)을 1.74배 감소시킴
- 연산자 선점 오버헤드를 최소화함
- 유연한 워크로드 우선순위 설정을 지원함
- 다양한 스케줄러 타임 슬라이스 설정에서 작동함
- 다양한 벡터 메모리 용량에서 이득을 얻음
- 여러 워크로드 및 SAVU 확장성을 가짐